

Лабораторна робота № 2

****Тема: **Наївний баєсів класифікатор для аналізу тональності текстів**

Мета роботи: Навчитися реалізовувати та використовувати наївний баєсів класифікатор для аналізу тональності текстів, оцінювати його ефективність.

Завдання:

1. Завантажити та попередньо обробити дані рецензій за допомогою функції `movie_review`.
2. Побудувати словник частотності слів для позитивного і негативного класів.
3. Обчислити логарифмічні апіорні ймовірності (*log prior*) для кожного класу.
4. Обчислити логарифмічні правдоподібності (*log likelihood*) для кожного слова і класу.
5. Реалізувати функцію наївного баєсового класифікатора.
6. Оцінити точність класифікатора на тестовому наборі даних.
7. Проаналізувати найбільш позитивні та негативні слова за співвідношенням їх частотностей.
8. Проаналізувати помилки класифікації.
9. Протестувати класифікатор на власному твіті. Розгорніть відповідну сторінку на [GitHub](#).
10. Завантажити код та результати на [GitHub](#), надати посилання на них та сторінку у [Moodle](#).

1. Завантажити та попередньо обробити дані рецензій за допомогою функції `movie_review`.

```
import nltk
import numpy as np
import pandas as pd
```

```
from nltk.corpus import movie_reviews
#from utils import process_reviews, build_freqs
```

```
nltk.download('movie_reviews')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data] Package movie_reviews is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
True
```

```
pos_fileids = movie_reviews.fileids('pos')
neg_fileids = movie_reviews.fileids('neg')

all_positive_reviews = [movie_reviews.raw(fileid) for fileid in pos_fileids]
all_negative_reviews = [movie_reviews.raw(fileid) for fileid in neg_fileids]

total_positive_lenght = len(all_positive_reviews)
total_negative_lenght = len(all_negative_reviews)

percent= 0.80

train_positive_count = round(total_positive_lenght * percent)
train_negative_count = round(total_negative_lenght * percent)

test_pos = all_positive_reviews[train_positive_count:]
train_pos = all_positive_reviews[:train_positive_count]
test_neg = all_negative_reviews[train_negative_count:]
train_neg = all_negative_reviews[:train_negative_count]

train_x = train_pos + train_neg
test_x = test_pos + test_neg

train_y = np.append(np.ones(len(train_pos)), np.zeros(len(train_neg)))
test_y = np.append(np.ones(len(test_pos)), np.zeros(len(test_neg)))
```

```
import re
import string

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import TweetTokenizer
```

```
from matplotlib.patches import Ellipse
import matplotlib.transforms as transforms

def process_review(review):
    """
    Очищення тексту, токенизація, видалення стоп-слів та стемінг
    """

    stemmer = PorterStemmer()
    stopwords_english = stopwords.words('english')
    review = re.sub(r'\\$\\w*', '', review)
    review = re.sub(r'^RT[\\s]+', '', review)
    review = re.sub(r'https?:/[\\s\\n\\r]+', '', review)
    review = re.sub(r'#', '', review)
    tokenizer = TweetTokenizer(
        preserve_case=False, strip_handles=True, reduce_len=True
    )
    review_tokens = tokenizer.tokenize(review)

    reviews_clean = []
    for word in review_tokens:
        if (
            word not in stopwords_english
            and any(ch.isalpha() for ch in word)
        ):
            stem_word = stemmer.stem(word)
            reviews_clean.append(stem_word)

    return reviews_clean
```

```
custom_review = "This movie was AMAZING!!! Best acting ever, totally loved it. Check review at http://imdb.com #cinema #mu:
print(process_review(custom_review))

['movi', 'amaz', 'best', 'act', 'ever', 'total', 'love', 'check', 'review', 'cinema', 'mustwatch', 'budget']
```

2. Побудувати словник частотності слів для позитивного і негативного класів

Алгоритм:

- Для кожної рецензії у тренувальному наборі:
 - Розбити рецензію на слова з допомогою process_review.
 - Для кожного слова:
 - Створити ключ - кортеж (слово, мітка класу).
 - Якщо ключ вже є в словнику, збільшити його значення на 1.
 - Інакше додати ключ до словника зі значенням 1.

```
def count_reviews(result, reviews, ys):
    for y, review in zip(ys, reviews):
        for word in process_review(review):
            pair = (word, y)

            if pair in result:
                result[pair] += 1
            else:
                result[pair] = 1

    return result
```

```
freqs = count_reviews({}, train_x, train_y)
```

```
len(freqs)
```

```
43376
```

3-4. Обчислити логарифмічні апіорні ймовірності (log prior) та правдоподібності (log likelihood).

Алгоритм:

- Порахувати кількість унікальних слів V.
- Порахувати загальну кількість позитивних слів N_pos і негативних слів N_neg.
- Порахувати кількість документів D.
- Порахувати кількість позитивних документів D_pos і негативних документів D_neg.

- Обчислити log prior: $\text{logprior} = \log(D_{\text{pos}}) - \log(D_{\text{neg}})$
- Для кожного слова у словнику:
 - Дістати частотності слова для позитивного і негативного класів.
 - Обчислити ймовірності слова для кожного класу з лапласівським згладжуванням: $p_{w_pos} = (\text{freq_pos} + 1) / (N_{\text{pos}} + V)$
 $p_{w_neg} = (\text{freq_neg} + 1) / (N_{\text{neg}} + V)$
 - Обчислити log likelihood як логарифм відношення ймовірностей:
 $\text{loglikelihood} = \log(p_{w_pos} / p_{w_neg})$

```
def train_naive_bayes(freqs, train_x, train_y):
    loglikelihood = {}
    logprior = 0

    vocab = set([pair[0] for pair in freqs.keys()])
    V = len(vocab)

    N_pos = N_neg = 0
    for pair in freqs.keys():
        if pair[1] > 0:
            N_pos += freqs[pair]
        else:
            N_neg += freqs[pair]

    D = len(train_y)
    D_pos = sum(train_y)
    D_neg = D - D_pos

    logprior = np.log(D_pos) - np.log(D_neg)

    for word in vocab:
        freq_pos = freqs.get((word, 1), 0)
        freq_neg = freqs.get((word, 0), 0)

        p_w_pos = (freq_pos + 1) / (N_pos + V)
        p_w_neg = (freq_neg + 1) / (N_neg + V)

        loglikelihood[word] = np.log(p_w_pos) - np.log(p_w_neg)

    return logprior, loglikelihood
```

```
logprior, loglikelihood = train_naive_bayes(freqs, train_x, train_y)
```

```
logprior
```

```
np.float64(0.0)
```

```
loglikelihood
```

```

danny': np.float64(0.5887119092279995),
'holbrook': np.float64(-0.7975824518918913),
'savant': np.float64(0.5887119092279995),
'stuff--except': np.float64(0.5887119092279995),
'crowd-pleas': np.float64(1.281859089787945),
'gentlemen': np.float64(-0.9517331317191484),
'theater': np.float64(-0.7975824518918913),
'cece': np.float64(0.5887119092279995),
'blake': np.float64(-0.2221830698832967),
'snorkel': np.float64(-0.7975824518918913),
'handmaid': np.float64(-0.7975824518918913),
'stivaletti': np.float64(0.5887119092279995),
'diarist': np.float64(0.5887119092279995),
'fifty-on': np.float64(0.5887119092279995),
'pee': np.float64(-0.7975824518918895),
'ozone': np.float64(0.5887119092279995),
'pan-and-scanned-out': np.float64(0.5887119092279995),
'kimono': np.float64(-0.7975824518918913),
'fish-out-wat': np.float64(-0.7975824518918913),
'dead-bang': np.float64(-0.7975824518918913),
'nature-lov': np.float64(0.5887119092279995),
'montague': np.float64(0.9941770173361633),
'gaelic': np.float64(0.9941770173361633),
'laughter': np.float64(-0.5293184652972123),
...}

```

5. Реалізувати функцію наївного баєсового класифікатора (функція класифікації рецензій)

Алгоритм:

- Обробити рецензію з допомогою process_review.
- Встановити початкову ймовірність $p = \text{logprior}$.
- Для кожного слова у рецензії:
 - Якщо слово є в словнику loglikelihood:
 - Додати loglikelihood цього слова до p .
- Повернути суму ймовірностей p .

```

def naive_bayes_predict(review, logprior, loglikelihood):
    word_l = process_review(review)

    p = logprior

    for word in word_l:
        if word in loglikelihood:
            p += loglikelihood[word]

    return p

```

```

if naive_bayes_predict("""When you look at the first two Tom Holland-starrer Spider-Man movies, they had a breezy feel simi
    logprior, loglikelihood) > 0:
    print ('позитивний')
else:
    print ('негативний')

```

позитивний

```

if naive_bayes_predict("""This saccharine, reverential biopic about controversial music legend Michael Jackson is set to be
    logprior, loglikelihood) > 0:
    print ('позитивний')
else:
    print ('негативний')

```

негативний

6. Оцінити точність класифікатора на тестовому наборі даних

Алгоритм:

- Для кожної рецензії в тестовому наборі:
 - Класифікувати рецензію з допомогою naive_bayes_predict.
 - Якщо ймовірність > 0 , присвоїти рецензії мітку 1, інакше 0.
 - Додати передбачену мітку до списку.
- Порахувати середню абсолютну похибку між передбаченими та справжніми мітками.
- Обчислити точність як $1 - \text{похибка}$.

```

def test_naive_bayes(test_x, test_y, logprior, loglikelihood):
    accuracy = 0

```

```

y_hats = []
for review in test_x:
    if naive_bayes_predict(review, logprior, loglikelihood) > 0:
        y_hat_i = 1
    else:
        y_hat_i = 0
    y_hats.append(y_hat_i)

error = np.mean(np.absolute(y_hats - test_y))
accuracy = 1 - error

return accuracy

```

```
test_naive_bayes(test_x, test_y, logprior, loglikelihood)
```

```
np.float64(0.795)
```

7. Проаналізувати найбільш позитивні та негативні слова за співвідношенням їх частотностей

```

def lookup(freqs, word, label):
    "Пошук скільки разів слово зустрілось у рецензіях певного класу"
    n = 0
    pair = (word, label)
    if pair in freqs:
        n = freqs[pair]

    return n

```

```

def get_ratio(freqs, word):
    """Відношення частотності слова в позитивному класі до негативного."""
    pos_neg_ratio = {'positive': 0, 'negative': 0, 'ratio': 0.0}

    pos_neg_ratio['positive'] = freqs.get((word, 1), 0)
    pos_neg_ratio['negative'] = freqs.get((word, 0), 0)
    pos_neg_ratio['ratio'] = (pos_neg_ratio['positive'] + 1) / (pos_neg_ratio['negative'] + 1)

    return pos_neg_ratio

```

```

def get_words_by_threshold(freqs, label, threshold, min_count=20):
    """Слова, чие відношення частотностей >= threshold (label=1) або <= threshold (label=0)."""
    word_list = {}

    for word in {w for w, _ in freqs}:
        pos_neg_ratio = get_ratio(freqs, word)

        if pos_neg_ratio['positive'] + pos_neg_ratio['negative'] < min_count:
            continue

        if label == 1 and pos_neg_ratio['ratio'] >= threshold:
            word_list[word] = pos_neg_ratio
        elif label == 0 and pos_neg_ratio['ratio'] <= threshold:
            word_list[word] = pos_neg_ratio

    return word_list

```

```

pos_words = get_words_by_threshold(freqs, 1, 3)

print("Найпозитивніші слова:")
for w, r in sorted(pos_words.items(), key=lambda x: x[1]['ratio'], reverse=True)[:15]:
    print(w, r)

```

```

Найпозитивніші слова:
shrek {'positive': 59, 'negative': 0, 'ratio': 60.0}
mulan {'positive': 78, 'negative': 1, 'ratio': 39.5}
gattaca {'positive': 36, 'negative': 0, 'ratio': 37.0}
guido {'positive': 31, 'negative': 0, 'ratio': 32.0}
flynt {'positive': 62, 'negative': 1, 'ratio': 31.5}
ordel {'positive': 30, 'negative': 0, 'ratio': 31.0}
leila {'positive': 29, 'negative': 0, 'ratio': 30.0}
sweetback {'positive': 27, 'negative': 0, 'ratio': 28.0}
taran {'positive': 26, 'negative': 0, 'ratio': 27.0}
homer {'positive': 26, 'negative': 0, 'ratio': 27.0}
mallori {'positive': 24, 'negative': 0, 'ratio': 25.0}
donkey {'positive': 22, 'negative': 0, 'ratio': 23.0}
rounder {'positive': 22, 'negative': 0, 'ratio': 23.0}
gile {'positive': 21, 'negative': 0, 'ratio': 22.0}
argento {'positive': 21, 'negative': 0, 'ratio': 22.0}

```

```
neg_words = get_words_by_threshold(freqs, 0, 1/3)

print("\nНайнегативніші слова:")
for w, r in sorted(neg_words.items(), key=lambda x: x[1]['ratio'][:15]):
    print(w, r)
```

```
Найнегативніші слова:
nbsp {'positive': 0, 'negative': 58, 'ratio': 0.01694915254237288}
psychlo {'positive': 0, 'negative': 35, 'ratio': 0.027777777777777776}
seagal {'positive': 0, 'negative': 30, 'ratio': 0.03225806451612903}
joli {'positive': 0, 'negative': 30, 'ratio': 0.03225806451612903}
brenner {'positive': 0, 'negative': 22, 'ratio': 0.043478260869565216}
tomb {'positive': 0, 'negative': 20, 'ratio': 0.047619047619047616}
pokemon {'positive': 0, 'negative': 20, 'ratio': 0.047619047619047616}
atroci {'positive': 0, 'negative': 20, 'ratio': 0.047619047619047616}
werewolf {'positive': 1, 'negative': 28, 'ratio': 0.06896551724137931}
schumach {'positive': 2, 'negative': 42, 'ratio': 0.06976744186046512}
8mm {'positive': 2, 'negative': 40, 'ratio': 0.07317073170731707}
jakob {'positive': 1, 'negative': 25, 'ratio': 0.07692307692307693}
magoo {'positive': 1, 'negative': 25, 'ratio': 0.07692307692307693}
jawbreak {'positive': 1, 'negative': 24, 'ratio': 0.08}
hudson {'positive': 1, 'negative': 23, 'ratio': 0.08333333333333333}
```

8. Проаналізувати помилки класифікації

Алгоритм:

- Для кожної пари (рецензія, справжня мітка) з тестового набору:
 - Класифікувати рецензію та отримати передбачену ймовірність.
 - Якщо справжня мітка не збігається з передбаченою:
 - Вивести справжню мітку, передбачену мітку і оброблену рецензію.

```
print('Truth Predicted Tweet')

errors = []
for x, y in zip(test_x, test_y):
    score = naive_bayes_predict(x, logprior, loglikelihood)
    y_hat = 1 if score > 0 else 0

    if y != y_hat:
        errors.append((x, y, y_hat, score))
        print('%d\t%d\t%s' % (y, y_hat, ' '.join(process_review(x)[:25])))

print(f'\nПомилки: {len(errors)} із {len(test_x)}')
```

```
1      0      start movi remind part movi stargat peopl look around egyptian templ read danger thing go destroy earth fu
1      0      gere willi poitier chase around world jackal film review michael redman copyright michael redman soviet un
1      0      usual blockbust come load effect star bad script plenti action mysteri men may contain all-star cast efect
1      0      get combin clueless dumb dumber answer question get director david mirkin' new comedi romi michele' high s
1      0      plot dude brother drive cross-countri decid fool around trucker cb radio long littl prank get someone put c
1      0      wyatt earp lot tell littl say stori legendari lawman run three hour nine minut that' long epic-wannab play
1      0      one thing bother hollywood film predict end devil' advoc atyp hollywood end everyth happen unfortun devil'
1      0      armageddon symbol everyth wrong modern filmmak stori replac special effect charact develop get overshadow
1      0      meteor threat set blow away volcano twister summer season could probabl ambiti season decad hollywood chur
1      0      director domin sena made highli underr kalifornia produc jerri bruckheim rock armageddon bring us slick en
1      0      got think twice go see movi titl like maximum risk titl gener meaningless can't believ good busi pick phon
```

```

0      1      movi pre-releas buzz insist nigh oscar potenti final releas everyon goe along thu sometim film unworthi aw
0      1      depend degre cinemat acumen last man stand either prohibition-era remak akira kurosawa' classic yojimbo b
0      1      rene zellweg star sonia young jewish wife mother frustrat constraint hasid commun brooklyn husband glenn f
0      1      let movi fool believ romant noir william shakespeare one truli understand heart soul man except work movi m
0      1      walken star mobster kidnap held ransom four bratti rich kid seem woman also kidnapped--sh sister one e hen
0      1      frank detorri' bill murray singl dad live beer junk food appar understand sanit hygien much dismay preteen
0      1      star war episod phantom menac director georg luca cast liam neeson ewan mcgregor natali portman jake lloyd
0      1      live older peopl twilight year attempt come grip share histori possibl futur fascin topic find all-star ca
0      1      sad state affair back box blurb excit movi contain within case paul mayersberg film _the last samurai thou
0      1      set harlem great depress rival gangster famili go war control number illeg gambl lotteri runner take bet p
0      1      think nostalgia seventi bad enough realli need eighti film robbi hart adam sandler use want rock roll star
0      1      new entri revisionist histori genr filmmak dick suggest two not-too-bright teenag girl caus uncov nation'
0      1      absolut power new film produc direct clint eastwood attempt thriller set world hypocrit presid murder poli
0      1      crown jewel irwin allen disast movi poseidon adventur featur all-star cast includ gene hackman ernest borg
0      1      bicentenni man famili film without extern motiv except provid minimum dose entertain chri columbu director
0      1      long ago film construct strong dialogu origin charact memor plot point solid act one best exampl hollywood
0      1      anna king least fourth film adapt margaret landon' fact base novel king big expans soulless though good-lo
0      1      king warner brother anim music featur recycl classic stori woman challeng heart king obviou result anna mi

```

Алгоритм:

- Для перших кількох помилок:
 - Обчислити внесок кожного слова в score (loglikelihood слова × кількість його повторів у рецензії).
 - Вивести слова з найбільшим за модулем внеском.
 - Пояснити причину помилки (сарказм, заперечення, змішана оцінка тощо).

```

def top_words(review, n=5):
    """Практична демонстрація того, чому модель помилилась"""
    words = process_review(review)
    contrib = {w: loglikelihood[w] * words.count(w) for w in set(words) if w in loglikelihood}
    return sorted(contrib.items(), key=lambda x: abs(x[1]), reverse=True)[:n]

for x, y, y_hat, score in errors[:5]:
    print(f'Истина: {int(y)} | Передбачено: {y_hat} | Score: {score:.2f}')
    print(x[:300].replace('\n', ' '), '...')
    print('Найвпливовіші слова:', top_words(x))
    print('-' * 80)

```

```

Истина: 1 | Передбачено: 0 | Score: -7.06
you'd think it would be easy to spoof star trek , but try spoofing it without cruelly mocking it . that's a bit more diffic
Найвпливовіші слова: [('trek', np.float64(4.648885133171163)), ('nesmith', np.float64(-3.9879122594594563)), ('gwen', np.flo
-----
Истина: 1 | Передбачено: 0 | Score: -17.53
in my review of there's something about mary , i said something to the effect of " laughter isn't everything . " i now need
Найвпливовіші слова: [('laughter', np.float64(-3.705229257080486)), ('mockeri', np.float64(3.3746483957922173)), ('nonsens',
-----
Истина: 1 | Передбачено: 0 | Score: -27.29
showgirls is the second major outing for the production team of pual verhoeven , director , and joe eszterhas , writer . th
Найвпливовіші слова: [('nomi', np.float64(-20.018644353042532)), ('showgirl', np.float64(-8.264405682869)), ('berkley', np.f
-----
Истина: 1 | Передбачено: 0 | Score: -23.71
one of the biggest cliches of any serial killer film is also one of the most believable . you know , the one where the dete
Найвпливовіші слова: [('mctiernan', np.float64(-6.904979546004492)), ('kiss', np.float64(-6.4893739582501055)), ('judd', np.
-----
Истина: 1 | Передбачено: 0 | Score: -2.82
`strange days' chronicles the last two days of 1999 in los angeles . as the locals gear up for the new millenium , lenny ne
Найвпливовіші слова: [('uninvolv', np.float64(-2.455810528495423)), ('kathryn', np.float64(-2.0503454203872575)), ('ambiti',
-----

```

9. Протестувати класифікатор на власній рецензії

```

my_review1 = """To say that The Love Hypothesis is my most anticipated movie in recent memory feels like an understatement.
In short: holy shit, yes. The Love Hypothesis delivered in every conceivable way. It is a faithful adaptation crafted with s
For those unfamiliar with the plot of Hazelwood's debut novel, the story follows Olive Smith, a doctoral candidate who begin
In case you missed her in the criminally-underrated Look Both Ways, then The Love Hypothesis is proof that if anyone can del
And then there is her co-star Tom Bateman, whose casting was met with such a hilarious reaction in certain circles, I am fra
As the heart of the story, Reinhart and Bateman's chemistry was always going to be the selling point, and luckily for us the
In terms of adaptational changes, there are surprisingly few. The moments and the lines that mean so much to readers like my
One moment in particular that I want to shout out, as I did in my review of the book five years ago (I cannot believe it's b
I hope The Love Hypothesis does well enough that it tells the powers that be that the people long for a rom-com. And not jus
"""

p = naive_bayes_predict(my_review1, logprior, loglikelihood)
print(p)

if p > 0:
    print('позитивний')
else:
    print('негативний')

print(top_words(my_review1))

```

```
15.78799009509559
позитивний
[('bateman', np.float64(4.921028172259181)), ('anh', np.float64(3.845577269363835)), ('adam', np.float64(-3.8264025056999778
```

```
my_review2 = """The Devil Wears Prada 2 is a dull, lifeless sequel that has no reason to exist. The original was sharp, fur

The worst part is how little the film does with its characters. Miranda Priestly, once a terrifying and brilliant villain,

The fashion, which should be the main attraction, is bland and disappointing. Instead of bold style we get endless product

At nearly two hours, it feels endless. I wanted to leave halfway through. Skip this disappointing mess and watch the origir
p = naive_bayes_predict(my_review2, logprior, loglikelihood)
print(p)

if p > 0:
    print('позитивний')
else:
    print('негативний')

print(top_words(my_review2))
```

```
-34.02786917885063
негативний
[('bore', np.float64(-2.5967154796087595)), ('miranda', np.float64(2.0927893060042724)), ('wast', np.float64(-1.783606521786
```

Класифікувати неоднозначні тексти, де позитивні та негативні оцінки співіснують, досить складно, бо наївний баєсів класифікатор не враховує контекст і загальний тон автора. Через це доводиться вводити додаткові покращення: змінювати поріг класифікації (замість 0 використовувати інше значення - експериментально шукати), розширювати попередню обробку або застосовувати складніші моделі.

Один із прикладів неоднозначного тексту є:

```
my_review3 = """“One Night Only,” director-producer Will Gluck’s latest romantic comedy, is everything that’s troubling about

Allie is a shy singer who makes a living belting out jingles for pharmaceutical products that sound like pop songs. Owen own

Unfortunately, Travis Braun’s script is nothing but an edgy premise—think Jac Schaeffer’s “TiMER” (2009) mixed with “The Pur

Good satire is rooted in the understanding and interrogation of systems, of the root causes of its satirical target. This ca

It’s a film all about sex that has no sense of eros between its central star-crossed would-be lovers, and only features the

Romance, too, is depicted in deeply conservative, heteronormative terms—casual sex is looked down on, open relationships onl

Sex workers are thrown under the bus, shown only as thieves who fleece men in the woods at night, while queer sexuality is r

This is supposed to be a nod to queer resilience, but largely comes off as straightwashing the history of anti-LGBTQ legisla

By the end, our intrepid protagonists have not had sex with anyone, because this is a film about Romance with a Capital R (a

And perhaps that’s the point of the movie; that we’re already this deep into a fascist state of emergency, yet only a few pe

You might ask, does a rom-com fail just because it refuses to actually engage with its political premise? Maybe not. But if

In one sequence where Owen and Allie can’t manage to pull $200 out of a bodega ATM in order to buy the lone condom left in a

If this is what it takes to get a rom-com greenlit these days, maybe we should just let them die. Death with dignity is sure
"""
p = naive_bayes_predict(my_review3, logprior, loglikelihood)
print(p)

if p > 10:
    print('позитивний')
else:
    print('негативний')

print(top_words(my_review3))
```

```
9.88684430875582
негативний
[('polit', np.float64(4.593310788653692)), ('fascist', np.float64(3.97670806934466)), ('conserv', np.float64(3.1390823244794
```

```
import json

data = {
    'logprior': float(logprior),
    'loglikelihood': {w: float(v) for w, v in loglikelihood.items()},
```

```
'stopwords': stopwords.words('english')
}

with open('naive_bayes_model.json', 'w') as outfile:
    json.dump(data, outfile)

from google.colab import files
files.download('naive_bayes_model.json')
```